Advanced Deployment Workshop

2026-04-21

Introduction

This workshop provides a comprehensive exploration of how Kubernetes manages pod lifecycle, from initial scheduling through eviction, and how container images are distributed across your cluster. You will learn the mechanisms that control where pods run, when they are moved or terminated, and how to configure your cluster for optimal performance and reliability.

Overview

The Advanced Deployment Lab covers four critical areas of Kubernetes operations:

- **Module 1 - Pod Scheduling Fundamentals**: Understanding the Kubernetes scheduling process, including the filtering and scoring phases, and how to use secondary scheduler operators for specialized workload requirements.
- **Module 2 - Advanced Pod Scheduling**: Learning advanced scheduling mechanisms including node affinity, taints and tolerations, Node Feature Discovery, pod affinity and anti-affinity, topology spread constraints with skew values, resource requirements, and pod priority with preemption.
- **Module 3 - Pod Descheduling and Scaling**: Exploring descheduling strategies, horizontal pod autoscaling (HPA), vertical pod autoscaling (VPA), and how these components interact with each other.
- **Module 4 - Image Distribution and Optimization**: Understanding image pull policies, image placement strategies, lean base images, and the Image Puller operator for optimizing image distribution across your cluster.

Prerequisites

This workshop assumes the following:

- Basic understanding of Kubernetes concepts (pods, nodes, namespaces)
- Familiarity with YAML configuration files
- Access to a Kubernetes cluster (version 1.20 or later recommended)
- Command-line experience with `oc`
- Understanding of container images and registries

If you are new to Kubernetes, we recommend reviewing the Kubernetes Concepts documentation before proceeding.

Learning Objectives

After completing this workshop, you will be able to:

- Understand the Kubernetes scheduling process, including filtering and scoring phases
- Configure and use secondary scheduler operators for specialized workloads

- Apply advanced scheduling mechanisms: node affinity, taints/tolerations, and topology spread constraints
- Leverage Node Feature Discovery labels for hardware-aware scheduling
- Configure pod priority classes and understand preemption behavior
- Understand pod descheduling strategies and the Descheduler Operator
- Configure horizontal and vertical pod autoscaling
- Recognize how HPA, VPA, and Descheduler can interact and potentially conflict
- Optimize image distribution strategies using image pull policies and the Image Puller operator
- Choose appropriate base images and understand their impact on deployment performance
- Diagnose and resolve scheduling failures
- Implement best practices for pod scheduling, scaling, and resource management

Module 1: Pod Scheduling Fundamentals

Content

The Kubernetes scheduler is responsible for assigning pods to nodes in your cluster. Understanding how scheduling works is fundamental to managing deployments effectively. This module explores the scheduling process and how to use secondary schedulers for specialized workload requirements.

The Scheduling Process

When you create a pod, it enters a `Pending` state until the scheduler assigns it to a node. The scheduling process consists of two main phases:

Filtering Phase

During the filtering phase, the scheduler evaluates all nodes and filters out those that cannot host the pod. A node is filtered out if:

- It lacks sufficient resources (CPU, memory, storage)
- It has taints that the pod cannot tolerate
- It doesn't match node selectors or node affinity rules
- It has other constraints that prevent pod placement

Scoring Phase

After filtering, the scheduler scores each remaining node based on various factors:

- Resource availability and balance
- Pod affinity and anti-affinity preferences
- Node preferences and topology constraints
- Inter-pod affinity requirements

The node with the highest score is selected for pod placement.

Secondary Scheduler Operator

The Secondary Scheduler Operator enables you to deploy and manage additional schedulers alongside the default Kubernetes scheduler. This allows you to create specialized schedulers with custom policies and behaviors for different workload types.

Understanding Secondary Schedulers

Secondary schedulers run in parallel with the default scheduler, each with their own scheduling policies and configurations. Pods can specify which scheduler to use via the `schedulerName` field in their pod specification.

Benefits of using secondary schedulers include:

- **Workload-Specific Policies**: Different schedulers can have different scoring algorithms and filters
- **Isolation**: Critical workloads can use dedicated schedulers with guaranteed resources
- **Custom Logic**: Implement domain-specific scheduling logic without modifying the default scheduler
- **Testing**: Test new scheduling policies without affecting the default scheduler

How Secondary Schedulers Work

When a pod specifies a secondary scheduler name, the Kubernetes API server routes the pod to that scheduler instead of the default one. Each scheduler operates independently:

1. **Pod Creation**: Pod is created with `schedulerName` specified
2. **Scheduler Selection**: API server routes pod to the specified scheduler
3. **Scheduling Process**: Secondary scheduler evaluates nodes using its own policies
4. **Node Binding**: Scheduler binds pod to selected node
5. **Pod Execution**: Kubelet on the selected node starts the pod

Using Secondary Schedulers

To use a secondary scheduler, specify the scheduler name in your pod or deployment:

```
apiVersion: v1
kind: Pod
metadata:
  name: secondary-scheduled-pod
spec:
  schedulerName: my-custom-scheduler
  containers:
  - name: app
    image: myapp:latest
```

For deployments:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
```

```
    metadata:
      labels:
        app: web
    spec:
      schedulerName: my-custom-scheduler
      containers:
      - name: app
        image: myapp:latest
```

Secondary Scheduler Configuration

Secondary schedulers can be configured with custom profiles, plugins, and scoring strategies. The configuration is typically managed through ConfigMaps or the scheduler's deployment configuration.

Example scheduler configuration:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: secondary-scheduler-config
  namespace: secondary-scheduler
data:
  config.yaml: |
    apiVersion: kubescheduler.config.k8s.io/v1
    kind: KubeSchedulerConfiguration
    profiles:
    - schedulerName: my-custom-scheduler
      plugins:
        score:
          enabled:
          - name: NodeResourcesFit
            weight: 2
          - name: NodeAffinity
            weight: 3
          - name: PodTopologySpread
            weight: 1
        filter:
          enabled:
          - name: NodeResourcesFit
          - name: NodeAffinity
          - name: TaintToleration
      pluginConfig:
      - name: NodeResourcesFit
        args:
          scoringStrategy:
            type: MostAllocated
            resources:
            - name: cpu
              weight: 1
```

```
            - name: memory
              weight: 1
```

Use Cases for Secondary Schedulers

High-Priority Workloads

Create a dedicated scheduler for high-priority workloads with aggressive resource allocation:

```
apiVersion: v1
kind: Pod
metadata:
  name: critical-pod
spec:
  schedulerName: high-priority-scheduler
  priorityClassName: high-priority
  containers:
  - name: app
    image: critical-app:latest
    resources:
      requests:
        cpu: "2"
        memory: "4Gi"
```

Batch Workloads

Use a secondary scheduler optimized for batch jobs with different scoring strategies:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: batch-job
spec:
  template:
    spec:
      schedulerName: batch-scheduler
      containers:
      - name: worker
        image: batch-worker:latest
      restartPolicy: Never
```

GPU Workloads

Dedicated scheduler for GPU workloads with specialized node selection:

```
apiVersion: v1
kind: Pod
metadata:
```

```
  name: gpu-workload
spec:
  schedulerName: gpu-scheduler
  containers:
  - name: gpu-app
    image: gpu-app:latest
    resources:
      limits:
        nvidia.com/gpu: 1
```

Verifying Scheduler Assignment

You can verify which scheduler handled a pod:

```
# Check pod scheduler name
oc get pod <pod-name> -o jsonpath='{.spec.schedulerName}'

# View scheduler events
oc describe pod <pod-name> | grep -i scheduler

# Check secondary scheduler logs
oc logs -n kube-system -l app=secondary-scheduler
```

Scheduler Selection Behavior

If a pod specifies a scheduler name that doesn't exist or isn't running:

- The pod remains in `Pending` state
- Events will indicate the scheduler is not available
- The default scheduler will not handle the pod automatically

Always ensure the specified scheduler is deployed and running before creating pods that reference it.

Best Practices for Secondary Schedulers

- Use secondary schedulers for workloads with specific scheduling requirements
- Ensure secondary schedulers are highly available (multiple replicas)
- Monitor secondary scheduler performance and resource usage
- Document which workloads use which schedulers
- Test scheduler configurations thoroughly before deploying to production
- Use descriptive scheduler names that indicate their purpose
- Consider scheduler resource requirements when planning cluster capacity

Pod Priority and Preemption

Pod priority and preemption allow you to influence the relative importance of pods and enable higher-priority pods to evict lower-priority pods when resources are constrained.

Understanding Pod Priority

Pod priority determines the relative importance of pods during scheduling and resource contention. Higher-priority pods are scheduled before lower-priority pods, and when resources are insufficient, lower-priority pods may be preempted (evicted) to make room for higher-priority pods.

Priority Classes

Priority classes define priority levels that can be assigned to pods:

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000
globalDefault: false
description: "High priority class for critical workloads"
```

Key parameters:

- **value**: The priority value (higher numbers = higher priority)
- **globalDefault**: If true, this priority class is used for pods without a priorityClassName
- **description**: Human-readable description of the priority class

Priority values can range from -2147483648 to 1000000000 (inclusive). System-level priority classes typically use values below 2000000000.

Using Priority Classes

Assign priority to pods by specifying a priority class:

```
apiVersion: v1
kind: Pod
metadata:
  name: high-priority-pod
spec:
  priorityClassName: high-priority
  containers:
  - name: app
    image: myapp:latest
    resources:
      requests:
        cpu: "500m"
```

```
        memory: "512Mi"
```

For deployments:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: critical-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: critical
  template:
    metadata:
      labels:
        app: critical
    spec:
      priorityClassName: high-priority
      containers:
      - name: app
        image: myapp:latest
        resources:
          requests:
            cpu: "500m"
            memory: "512Mi"
```

Preemption

Preemption occurs when a higher-priority pod cannot be scheduled because nodes lack sufficient resources. The scheduler may evict lower-priority pods to make room for the higher-priority pod.

Preemption behavior:

- Only pods with lower priority can be preempted
- Preemption respects Pod Disruption Budgets (PDBs)
- Preempted pods are gracefully terminated
- Preemption only occurs when necessary for scheduling higher-priority pods

Preemption Scenarios

Preemption can occur in several scenarios:

Insufficient Resources

When a high-priority pod requires resources that are currently allocated to lower-priority pods:

```
# Low priority pod running
```

```
apiVersion: v1
kind: Pod
metadata:
  name: low-priority-pod
spec:
  priorityClassName: low-priority
  containers:
  - name: app
    image: myapp:latest
    resources:
      requests:
        cpu: "2"
        memory: "4Gi"

# High priority pod that needs the same resources
apiVersion: v1
kind: Pod
metadata:
  name: high-priority-pod
spec:
  priorityClassName: high-priority
  containers:
  - name: app
    image: critical-app:latest
    resources:
      requests:
        cpu: "2"
        memory: "4Gi"
```

If the high-priority pod cannot be scheduled, the scheduler may preempt the low-priority pod.

Node Resource Constraints

When nodes are at capacity and a high-priority pod needs to be scheduled:

```
# Check node resource usage
oc describe nodes | grep -A 5 "Allocated resources"

# View pods by priority
oc get pods --all-namespaces --sort-by=.spec.priority
```

The scheduler evaluates which lower-priority pods can be preempted to accommodate the higher-priority pod.

Preemption Process

When preemption occurs:

1. **Evaluation**: Scheduler identifies lower-priority pods that can be preempted

2. **PDB Check**: Verifies that preemption won't violate Pod Disruption Budgets
3. **Graceful Termination**: Sends termination signal to preempted pods
4. **Resource Release**: Waits for resources to be freed
5. **Scheduling**: Schedules the higher-priority pod

Viewing Priority and Preemption

Monitor pod priority and preemption events:

```
# View pod priority
oc get pod <pod-name> -o jsonpath='{.spec.priorityClassName}'
oc get pod <pod-name> -o jsonpath='{.spec.priority}'

# View all priority classes
oc get priorityclasses

# View preemption events
oc get events --field-selector reason=Preempted

# Check for preempted pods
oc get pods --all-namespaces --field-selector=status.phase=Failed | grep Preempted
```

Priority Class Best Practices

- Use priority classes to distinguish critical workloads from best-effort workloads
- Create a small number of well-defined priority levels
- Document priority class usage and policies
- Avoid using `globalDefault: true` unless necessary
- Monitor preemption events to understand cluster behavior
- Use Pod Disruption Budgets to protect important workloads from preemption
- Test priority configurations in non-production environments
- Consider resource requirements when assigning priorities

Priority and Resource Requests

Priority works in conjunction with resource requests:

- Pods without resource requests cannot preempt pods with requests
- Priority alone doesn't guarantee scheduling; resources must be available
- Higher-priority pods still need sufficient resources to be scheduled
- Preemption only occurs when lower-priority pods can be evicted

Example:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: high-priority-no-requests
spec:
  priorityClassName: high-priority
  containers:
  - name: app
    image: myapp:latest
    # No resource requests - cannot preempt other pods
```

This pod has high priority but cannot preempt other pods because it has no resource requests.

Module 2: Advanced Pod Scheduling

Content

This module covers advanced scheduling mechanisms that allow fine-grained control over pod placement, resource allocation, and priority management in your cluster.

Node Selection Methods

Kubernetes provides several mechanisms to influence where pods are scheduled:

Node Selectors

Node selectors are the simplest way to constrain pod placement. They require nodes to have specific labels:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  nodeSelector:
    disktype: ssd
    zone: us-west-1
  containers:
  - name: nginx
    image: nginx:latest
```

Node Affinity

Node affinity provides more flexible scheduling rules than node selectors. It supports both required and preferred rules:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: zone
            operator: In
            values:
            - us-west-1
```

```
            - us-west-2
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
          - key: disktype
            operator: In
            values:
            - ssd
  containers:
  - name: nginx
    image: nginx:latest
```

The `requiredDuringSchedulingIgnoredDuringExecution` rule must be satisfied for the pod to be scheduled. The `preferredDuringSchedulingIgnoredDuringExecution` rule is a preference that influences scoring but doesn't prevent scheduling.

Taints and Tolerations

Taints allow nodes to repel pods that don't have matching tolerations. This is useful for reserving nodes for specific workloads:

```
# Add a taint to a node
oc taint nodes node1 key=value:NoSchedule

# Pods must have a matching toleration
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  tolerations:
  - key: "key"
    operator: "Equal"
    value: "value"
    effect: "NoSchedule"
  containers:
  - name: nginx
    image: nginx:latest
```

Taint effects include:
* `NoSchedule`: Pods without toleration cannot be scheduled
* `PreferNoSchedule`: Scheduler tries to avoid the node but doesn't guarantee it
* `NoExecute`: Existing pods without toleration are evicted

Node Feature Discovery

Node Feature Discovery (NFD) automatically detects hardware features and capabilities on cluster nodes and labels them accordingly. This enables you to schedule workloads on nodes with specific hardware features such as GPUs, specialized CPUs, or other accelerators.

NFD runs as a DaemonSet, with one pod per node that discovers and labels node features. Common features detected include:

- CPU features (AVX, SSE, specific instruction sets)
- Hardware accelerators (GPUs, FPGAs, specialized processors)
- Kernel features and modules
- PCI devices
- System topology information

How NFD Works

NFD consists of two main components:

- **Node Feature Discovery Master**: Runs as a Deployment, manages feature discovery and labeling
- **Node Feature Discovery Worker**: Runs as a DaemonSet, discovers features on each node and reports them

The worker discovers features using source plugins:

- **CPU plugin**: Detects CPU features and capabilities
- **Kernel plugin**: Discovers kernel features and loaded modules
- **PCI plugin**: Identifies PCI devices
- **System plugin**: Detects system-level features
- **Custom plugins**: Allow custom feature detection logic

Viewing Discovered Features

You can view the labels that NFD creates on nodes:

```
# View all node labels including NFD labels
oc get nodes --show-labels

# View specific node with NFD labels
oc describe node <node-name>

# Filter for NFD-specific labels
oc get nodes -o json | jq '.items[].metadata.labels | with_entries(select(.key |
startswith("feature.node.kubernetes.io")))'
```

NFD labels follow the pattern `feature.node.kubernetes.io/<feature-name>=<value>`.

Using NFD Labels for Scheduling

Once nodes are labeled with their features, you can use these labels in node selectors or node affinity rules:

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-workload
spec:
  nodeSelector:
    feature.node.kubernetes.io/cpu-cpuid.AVX512F: "true"
    feature.node.kubernetes.io/pci-0300_10de.present: "true"
  containers:
  - name: gpu-app
    image: nvidia/cuda:latest
    resources:
      limits:
        nvidia.com/gpu: 1
```

Or using node affinity for more flexible matching:

```
apiVersion: v1
kind: Pod
metadata:
  name: optimized-workload
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: feature.node.kubernetes.io/cpu-cpuid.AVX2
            operator: In
            values:
            - "true"
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
          - key: feature.node.kubernetes.io/kernel-version.major
            operator: Gt
            values:
            - "5"
  containers:
  - name: app
    image: myapp:latest
```

Custom Feature Sources

You can extend NFD with custom feature sources to detect application-specific features:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nfd-worker-config
  namespace: node-feature-discovery
data:
  custom-source.conf: |
    - name: "my-custom-feature"
      matchOn:
      - loadedKMod: ["my-module"]
      - pciId:
          class: ["03"]
          vendor: ["10de"]
```

NFD Best Practices

- Use NFD to automatically label nodes with hardware capabilities
- Combine NFD labels with resource requests for optimal scheduling
- Review and filter NFD labels to avoid label bloat
- Use node affinity with NFD labels for flexible feature matching
- Monitor NFD worker pods to ensure feature discovery is working correctly
- Consider using NFD with device plugins for specialized hardware

Pod Affinity and Anti-Affinity

Pod affinity rules allow you to specify how pods should be placed relative to other pods:

Pod Affinity

Pod affinity ensures pods are scheduled near other pods with matching labels:

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
```

```
          - key: app
            operator: In
            values:
            - cache
        topologyKey: kubernetes.io/hostname
  containers:
  - name: web
    image: nginx:latest
```

This pod will only be scheduled on nodes that have pods with `app=cache` label.

Pod Anti-Affinity

Pod anti-affinity ensures pods are not scheduled near other pods with matching labels:

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
            - key: app
              operator: In
              values:
              - web
          topologyKey: kubernetes.io/hostname
  containers:
  - name: web
    image: nginx:latest
```

This configuration prefers spreading pods with `app=web` across different nodes.

Topology Spread Constraints

Topology spread constraints provide fine-grained control over how pods are distributed across topology domains (such as nodes, zones, or regions). Unlike pod anti-affinity, topology spread constraints allow you to specify a maximum skew value that defines how uneven the distribution can be.

Understanding Skew Values

The `maxSkew` parameter defines the maximum difference in the number of pods between any two topology domains. A skew value of 0 means perfectly even distribution, while higher values allow for more uneven distribution.

Consider a deployment with 5 replicas across 3 zones:

- **maxSkew: 0**: Requires exactly even distribution (2, 2, 1 or similar)
- **maxSkew: 1**: Allows one pod difference between zones (2, 2, 1 or 3, 1, 1)
- **maxSkew: 2**: Allows two pod difference between zones (3, 1, 1 or 2, 2, 1)

Basic Topology Spread Constraint

```
apiVersion: v1
kind: Pod
metadata:
  name: spread-pod
  labels:
    app: web
spec:
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: web
  containers:
  - name: app
    image: myapp:latest
```

Key parameters:

- `maxSkew`: Maximum difference in pod count between topology domains
- `topologyKey`: Node label key that defines topology domains
- `whenUnsatisfiable`: Action when constraint cannot be satisfied (`DoNotSchedule` or `ScheduleAnyway`)
- `labelSelector`: Selects which pods to count for skew calculation

Node-Level Distribution

Spread pods evenly across nodes:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
```

```
  replicas: 6
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: web
      containers:
      - name: app
        image: myapp:latest
```

With 6 replicas and 3 nodes, this ensures each node gets 2 pods (maxSkew: 1 allows 2-2-2 or 3-2-1 distribution).

Zone-Level Distribution

Distribute pods across availability zones:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 9
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: web
      containers:
      - name: app
```

```
        image: myapp:latest
```

This distributes 9 replicas across zones with at most 1 pod difference between any two zones (e.g., 3-3-3 or 4-3-2).

Multiple Constraints

You can specify multiple topology spread constraints:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 12
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: web
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: web
      containers:
      - name: app
        image: myapp:latest
```

This ensures even distribution both across zones and across nodes within each zone.

Impact of Skew Values on Scheduling

The `maxSkew` value directly impacts scheduling behavior:

- **Low skew (0-1)**: Strict distribution requirements, may prevent scheduling if constraints cannot be met
- **Medium skew (2-3)**: Balanced distribution with some flexibility

- **High skew (4+)**: More permissive, allows significant imbalance

Example with 10 replicas and 3 zones:

```
# maxSkew: 1 - Very strict (4-3-3 or 4-4-2)
maxSkew: 1

# maxSkew: 2 - Moderate (4-4-2 or 5-3-2)
maxSkew: 2

# maxSkew: 3 - Permissive (5-3-2 or 6-2-2)
maxSkew: 3
```

WhenUnsatisfiable Behavior

The `whenUnsatisfiable` field controls what happens when constraints cannot be met:

- **DoNotSchedule**: Pod remains in Pending state if constraint cannot be satisfied
- **ScheduleAnyway**: Pod is scheduled anyway, but the scheduler still tries to minimize skew

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: ScheduleAnyway
        labelSelector:
          matchLabels:
            app: web
      containers:
      - name: app
        image: myapp:latest
```

Use `ScheduleAnyway` when you want best-effort distribution but don't want to block scheduling.

Combining with Other Constraints

Topology spread constraints work alongside other scheduling constraints:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 6
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      nodeSelector:
        disktype: ssd
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: web
      containers:
      - name: app
        image: myapp:latest
        resources:
          requests:
            memory: "256Mi"
            cpu: "100m"
```

The scheduler evaluates all constraints together, ensuring pods are scheduled on nodes that meet all requirements while respecting topology spread.

Resource Requirements

Resource requests and limits are critical for scheduling decisions:

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
  - name: app
```

```
    image: myapp:latest
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

- **Requests**: The scheduler uses these to determine if a node has sufficient resources. The pod is guaranteed at least these resources.
- **Limits**: The maximum resources a container can use. Exceeding limits may result in throttling or termination.

Scheduling Failures

When a pod cannot be scheduled, it remains in `Pending` state. Common reasons include:

- Insufficient resources on any node
- No nodes match node selectors or affinity rules
- All nodes have taints that the pod cannot tolerate
- Topology spread constraints cannot be satisfied (when `whenUnsatisfiable: DoNotSchedule`)
- Specified secondary scheduler is not available or not running
- Scheduler configuration issues

You can diagnose scheduling issues using:

```
# Check pod events
oc describe pod <pod-name>

# Check node resources
oc describe nodes

# Check scheduler logs
oc logs -n kube-system -l component=kube-scheduler
```

Best Practices

- Always specify resource requests and limits for predictable scheduling
- Use priority classes to distinguish critical workloads from best-effort workloads
- Use Pod Disruption Budgets to protect important workloads from preemption
- Use node affinity for placement requirements, not node selectors when possible
- Leverage pod anti-affinity to distribute workloads across nodes
- Use topology spread constraints with appropriate skew values for fine-grained distribution

control

- Choose skew values based on your availability requirements: lower skew for strict distribution, higher skew for flexibility
- Use taints and tolerations to reserve nodes for specific workloads
- Leverage Node Feature Discovery labels to identify nodes with hardware capabilities
- Use NFD labels with node selectors or affinity rules to schedule workloads on appropriate hardware
- Monitor scheduling metrics and preemption events to identify bottlenecks
- Test scheduling constraints and priority configurations in non-production environments first

Exercises

Exercise 1: Node Selectors and Node Affinity

In this exercise, you will practice using node selectors and node affinity to control pod placement.

Step 1: Examine Node Labels

First, let's see what labels are available on your cluster nodes:

```
# View all nodes and their labels
oc get nodes --show-labels

# Get detailed information about a specific node
oc describe node <node-name>
```

Step 2: Create a Pod with Node Selector

Create a pod that uses a node selector to target a specific node:

```
apiVersion: v1
kind: Pod
metadata:
  name: node-selector-pod
spec:
  nodeSelector:
    kubernetes.io/hostname: <node-name>
  containers:
  - name: nginx
    image: nginx:latest
```

Apply the pod and verify it's scheduled on the correct node:

```
oc apply -f node-selector-pod.yaml
```

```
oc get pod node-selector-pod -o wide
```

Step 3: Use Node Affinity

Create a pod with node affinity rules that are more flexible than node selectors:

```
apiVersion: v1
kind: Pod
metadata:
  name: node-affinity-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: kubernetes.io/arch
            operator: In
            values:
            - amd64
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
          - key: node-role.kubernetes.io/worker
            operator: Exists
  containers:
  - name: nginx
    image: nginx:latest
```

Apply and verify the pod placement:

```
oc apply -f node-affinity-pod.yaml
oc get pod node-affinity-pod -o wide
oc describe pod node-affinity-pod
```

Step 4: Clean Up

Remove the test pods:

```
oc delete pod node-selector-pod node-affinity-pod
```

Exercise 2: Taints and Tolerations

Practice using taints and tolerations to control which pods can run on specific nodes.

Step 1: Add a Taint to a Node

> Adding taints to nodes requires elevated privileges. Please wait for your instructor to add the taint before proceeding.

The instructor will add a taint to prevent regular pods from being scheduled on a node:

```
# Add a taint to a node (requires elevated privileges - instructor will run this)
oc taint nodes <node-name> special-workload=true:NoSchedule

# Verify the taint
oc describe node <node-name> | grep Taint
```

After the instructor has added the taint, verify it exists using the second command above.

Step 2: Attempt to Schedule a Regular Pod

Try to create a pod without a toleration:

```
apiVersion: v1
kind: Pod
metadata:
  name: regular-pod
spec:
  containers:
  - name: nginx
    image: nginx:latest
```

```
oc apply -f regular-pod.yaml
oc get pod regular-pod
oc describe pod regular-pod
```

Observe that the pod remains in `Pending` state due to the taint.

Step 3: Create a Pod with Toleration

Create a pod that can tolerate the taint:

```
apiVersion: v1
kind: Pod
metadata:
  name: tolerated-pod
spec:
  tolerations:
  - key: "special-workload"
    operator: "Equal"
    value: "true"
```

```
    effect: "NoSchedule"
  containers:
  - name: nginx
    image: nginx:latest
```

```
oc apply -f tolerated-pod.yaml
oc get pod tolerated-pod -o wide
```

Verify that this pod is scheduled on the tainted node.

Step 4: Remove the Taint

> Removing taints from nodes requires elevated privileges. Please ask your instructor to remove the taint when you're finished with this exercise.

The instructor will remove the taint:

```
# Remove the taint (requires elevated privileges - instructor will run this)
oc taint nodes <node-name> special-workload=true:NoSchedule-

# Clean up your test pods
oc delete pod regular-pod tolerated-pod
```

Exercise 3: Pod Affinity and Anti-Affinity

Practice using pod affinity and anti-affinity to control pod placement relative to other pods.

Step 1: Create a Deployment with Pod Anti-Affinity

Create a deployment that spreads pods across different nodes:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      affinity:
        podAntiAffinity:
```

```
          preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchExpressions:
                - key: app
                  operator: In
                  values:
                  - web
              topologyKey: kubernetes.io/hostname
      containers:
      - name: nginx
        image: nginx:latest
```

```
oc apply -f web-deployment.yaml
oc get pods -l app=web -o wide
```

Verify that pods are spread across different nodes.

Step 2: Create a Pod with Pod Affinity

Create a pod that prefers to be scheduled on the same node as the web pods:

```
apiVersion: v1
kind: Pod
metadata:
  name: cache-pod
  labels:
    app: cache
spec:
  affinity:
    podAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
            - key: app
              operator: In
              values:
              - web
          topologyKey: kubernetes.io/hostname
  containers:
  - name: redis
    image: redis:latest
```

```
oc apply -f cache-pod.yaml
```

```
oc get pod cache-pod -o wide
oc describe pod cache-pod
```

Check which node the cache pod was scheduled on and verify it's on the same node as at least one web pod.

Step 3: Clean Up

```
oc delete deployment web-deployment
oc delete pod cache-pod
```

Exercise 4: Topology Spread Constraints

Practice using topology spread constraints to control pod distribution with skew values.

Step 1: Create a Deployment with Topology Spread

Create a deployment with topology spread constraints:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spread-deployment
spec:
  replicas: 6
  selector:
    matchLabels:
      app: spread
  template:
    metadata:
      labels:
        app: spread
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: spread
      containers:
      - name: nginx
        image: nginx:latest
```

```
oc apply -f spread-deployment.yaml
oc get pods -l app=spread -o wide
```

Step 2: Analyze Pod Distribution

Count how many pods are on each node:

```
oc get pods -l app=spread -o wide | awk '{print $7}' | sort | uniq -c
```

Verify that the distribution respects the maxSkew value of 1.

Step 3: Experiment with Different Skew Values

Scale the deployment and observe how different skew values affect distribution:

```
# Scale to 10 replicas
oc scale deployment spread-deployment --replicas=10

# Check distribution
oc get pods -l app=spread -o wide | awk '{print $7}' | sort | uniq -c
```

Update the deployment to use maxSkew: 2 and observe the difference:

```
# Edit the deployment
oc edit deployment spread-deployment
# Change maxSkew from 1 to 2
```

```
# Check the new distribution
oc get pods -l app=spread -o wide | awk '{print $7}' | sort | uniq -c
```

Step 4: Clean Up

```
oc delete deployment spread-deployment
```

Exercise 5: Node Feature Discovery Labels

Practice using Node Feature Discovery labels to schedule pods on nodes with specific hardware features.

Step 1: View NFD Labels

Examine the Node Feature Discovery labels on your nodes:

```
# View all labels including NFD labels
oc get nodes --show-labels | grep feature.node.kubernetes.io

# Get detailed NFD labels for a specific node
```

```
oc describe node <node-name> | grep feature.node.kubernetes.io
```

Step 2: Create a Pod Using NFD Labels

Create a pod that uses NFD labels in a node selector:

```
apiVersion: v1
kind: Pod
metadata:
  name: nfd-pod
spec:
  nodeSelector:
    feature.node.kubernetes.io/cpu-cpuid.AVX: "true"
  containers:
  - name: app
    image: nginx:latest
```

```
oc apply -f nfd-pod.yaml
oc get pod nfd-pod -o wide
oc describe pod nfd-pod
```

Step 3: Use NFD Labels with Node Affinity

Create a pod using NFD labels with node affinity for more flexibility:

```
apiVersion: v1
kind: Pod
metadata:
  name: nfd-affinity-pod
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
          - key: feature.node.kubernetes.io/cpu-cpuid.AVX2
            operator: In
            values:
            - "true"
  containers:
  - name: app
    image: nginx:latest
```

```
oc apply -f nfd-affinity-pod.yaml
```

```
oc get pod nfd-affinity-pod -o wide
```

Step 4: Clean Up

```
oc delete pod nfd-pod nfd-affinity-pod
```

Exercise 6: Resource Requirements and Scheduling

Practice setting resource requests and limits and observe their impact on scheduling.

Step 1: Check Node Resources

Examine available resources on your nodes:

```
# View node resource capacity and allocatable
oc describe nodes | grep -A 5 "Allocated resources"

# Get resource usage summary
oc top nodes
```

Step 2: Create Pods with Resource Requests

Create a pod with specific resource requests:

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
  - name: app
    image: nginx:latest
    resources:
      requests:
        memory: "256Mi"
        cpu: "250m"
      limits:
        memory: "512Mi"
        cpu: "500m"
```

```
oc apply -f resource-pod.yaml
oc get pod resource-pod -o wide
oc describe pod resource-pod
```

Step 3: Create Multiple Pods and Observe Scheduling

Create several pods with resource requirements and observe how they're distributed:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      app: resource-app
  template:
    metadata:
      labels:
        app: resource-app
    spec:
      containers:
      - name: app
        image: nginx:latest
        resources:
          requests:
            memory: "128Mi"
            cpu: "100m"
          limits:
            memory: "256Mi"
            cpu: "200m"
```

```
oc apply -f resource-deployment.yaml
oc get pods -l app=resource-app -o wide
oc top nodes
```

Observe how the scheduler distributes pods based on available resources.

Step 4: Attempt to Overcommit Resources

Try to create a pod that requests more resources than available:

```
apiVersion: v1
kind: Pod
metadata:
  name: overcommit-pod
spec:
  containers:
  - name: app
    image: nginx:latest
    resources:
```

```
      requests:
        memory: "1000Gi"
        cpu: "1000"
```

```
oc apply -f overcommit-pod.yaml
oc get pod overcommit-pod
oc describe pod overcommit-pod
```

Observe that the pod remains in `Pending` state due to insufficient resources.

Step 5: Clean Up

```
oc delete pod resource-pod overcommit-pod
oc delete deployment resource-deployment
```

Exercise 7: Secondary Scheduler

Practice using a secondary scheduler for pod scheduling.

Step 1: Verify Available Schedulers

Check what schedulers are available in your cluster:

```
# List scheduler pods
oc get pods -n kube-system | grep scheduler

# Check scheduler configurations
oc get configmap -n kube-system | grep scheduler
```

Step 2: Create a Pod with Secondary Scheduler

Create a pod that specifies a secondary scheduler name:

```
apiVersion: v1
kind: Pod
metadata:
  name: secondary-scheduled-pod
spec:
  schedulerName: my-custom-scheduler
  containers:
  - name: app
    image: nginx:latest
```

```
oc apply -f secondary-scheduled-pod.yaml
oc get pod secondary-scheduled-pod
```

```
oc describe pod secondary-scheduled-pod
```

Note: If the secondary scheduler doesn't exist, the pod will remain in `Pending` state. Check the events to see the scheduling failure.

Step 3: Use Default Scheduler

Create the same pod without specifying a scheduler name (uses default):

```
apiVersion: v1
kind: Pod
metadata:
  name: default-scheduled-pod
spec:
  containers:
  - name: app
    image: nginx:latest
```

```
oc apply -f default-scheduled-pod.yaml
oc get pod default-scheduled-pod -o wide
oc describe pod default-scheduled-pod | grep -i scheduler
```

Step 4: Verify Scheduler Assignment

Check which scheduler handled each pod:

```
# Check scheduler name in pod spec
oc get pod secondary-scheduled-pod -o jsonpath='{.spec.schedulerName}'
oc get pod default-scheduled-pod -o jsonpath='{.spec.schedulerName}'
```

Step 5: Clean Up

```
oc delete pod secondary-scheduled-pod default-scheduled-pod
```

Module 3: Pod Descheduling and Scaling

Content

This module explores how Kubernetes manages pod lifecycle beyond initial scheduling, including descheduling pods to optimize cluster utilization, and automatic scaling mechanisms that adjust workloads based on demand. Understanding these mechanisms is crucial for maintaining efficient and responsive applications.

Understanding Descheduling

Descheduling is the process of removing pods from nodes to improve cluster balance and resource utilization. Unlike eviction, which is driven by resource pressure, descheduling is a proactive optimization mechanism.

Descheduling vs. Eviction

It's important to distinguish between descheduling and eviction:

- **Eviction**: Reactive removal of pods due to resource pressure or policy violations (covered in Module 1)
- **Descheduling**: Proactive removal of pods to optimize cluster balance and utilization

Descheduling typically moves pods to better-suited nodes, while eviction may terminate pods entirely.

Descheduling Behavior

> Descheduling does not happen by default in Kubernetes. The default scheduler only places pods; it does not move them after placement. To enable descheduling, you need a descheduler component that actively monitors and rebalances pods.

The descheduler evaluates running pods and identifies opportunities to improve cluster balance by:

- Moving pods from over-utilized nodes to under-utilized nodes
- Consolidating pods to free up nodes for maintenance
- Spreading pods more evenly across nodes
- Removing duplicate pods when anti-affinity rules are violated

Descheduler Operator Profiles

The Descheduler Operator provides nine predefined profiles that combine various descheduling strategies. Each profile is designed to address specific cluster optimization scenarios. Profiles are configured through the Descheduler custom resource as documented in the OpenShift Descheduler Operator repository.

The available profiles are:

- AffinityAndTaints
- TopologyAndDuplicates
- SoftTopologyAndDuplicates
- LifecycleAndUtilization
- LongLifecycle
- CompactAndScale
- KubeVirtRelieveAndMigrate
- EvictPodsWithPVC
- EvictPodsWithLocalStorage

Descheduler Operator Configuration

The Descheduler Operator is configured through a Descheduler custom resource. The operator runs as a controller that periodically evaluates the cluster and deschedules pods when conditions are met. Configuration typically includes:

- **Descheduling interval**: How often to evaluate pods for descheduling
- **Strategy parameters**: Thresholds and rules for each enabled strategy
- **Namespaces to include/exclude**: Scope of descheduling operations

Example Descheduler configuration using profiles:

```
apiVersion: operator.openshift.io/v1
kind: KubeDescheduler
metadata:
  name: cluster
  namespace: openshift-kube-descheduler-operator
spec:
  managementState: Managed
  deschedulingIntervalSeconds: 3600
  profiles:
  - LifecycleAndUtilization
  - RemovePodsViolatingInterPodAntiAffinity
  mode: Automatic
  evictionLimits:
    total: 10
    node: 5
```

Key configuration parameters:

- `profiles`: List of profile names to enable (choose from the nine available profiles)
- `deschedulingIntervalSeconds`: How often the descheduler runs
- `mode`: Either `Predictive` (simulation only) or `Automatic` (actual evictions)
- `evictionLimits`: Restrict the number of evictions per run

Monitoring Descheduling

Monitor descheduling operations to understand cluster optimization:

```
# View Descheduler Operator status
oc get descheduler

# View detailed Descheduler configuration
oc describe descheduler cluster-descheduler

# View descheduler operator logs
oc logs -n openshift-descheduler-operator -l app=descheduler-operator

# Check for descheduled pods
oc get events --field-selector reason=Descheduled

# Monitor pod movements
oc get pods -o wide --watch
```

Horizontal Pod Autoscaler (HPA)

Horizontal Pod Autoscaler automatically scales the number of pod replicas based on observed metrics such as CPU utilization, memory usage, or custom metrics.

How HPA Works

HPA monitors specified metrics and adjusts the replica count of a Deployment, StatefulSet, or other scalable resource:

1. **Metrics Collection**: HPA queries metrics from the Metrics API
2. **Target Calculation**: Compares current metrics to target values
3. **Replica Adjustment**: Increases or decreases replicas to meet targets
4. **Cooldown Periods**: Respects scale-up and scale-down delays to prevent thrashing

Creating an HPA

Create an HPA to automatically scale a deployment:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-deployment
  minReplicas: 2
```

```
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
      - type: Percent
        value: 50
        periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
      - type: Percent
        value: 100
        periodSeconds: 15
      - type: Pods
        value: 4
        periodSeconds: 15
      selectPolicy: Max
```

Key parameters:

- `minReplicas`: Minimum number of replicas to maintain
- `maxReplicas`: Maximum number of replicas allowed
- `metrics`: Metrics to monitor and target values
- `behavior`: Scaling behavior including stabilization windows and policies

HPA Metrics

HPA supports various metric types:

Resource Metrics

Scale based on CPU or memory utilization:

```
metrics:
- type: Resource
```

```
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
```

Custom Metrics

Scale based on custom application metrics:

```
metrics:
- type: Pods
  pods:
    metric:
      name: requests_per_second
    target:
      type: AverageValue
      averageValue: "100"
```

Object Metrics

Scale based on metrics from other Kubernetes objects:

```
metrics:
- type: Object
  object:
    metric:
      name: requests-per-second
    describedObject:
      apiVersion: networking.k8s.io/v1
      kind: Ingress
      name: main-route
    target:
      type: Value
      value: "2000"
```

HPA Behavior Configuration

Control how HPA scales with behavior settings:

```
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300
    policies:
    - type: Percent
      value: 50
      periodSeconds: 60
  scaleUp:
```

```
    stabilizationWindowSeconds: 0
    policies:
    - type: Percent
      value: 100
      periodSeconds: 15
    selectPolicy: Max
```

- `stabilizationWindowSeconds`: Time window to consider historical metrics
- `policies`: Scaling policies (Percent or Pods)
- `selectPolicy`: How to select among multiple policies (Max, Min, Disabled)

Monitoring HPA

Monitor HPA behavior and scaling decisions:

```
# View HPA status
oc get hpa web-hpa

# Get detailed HPA information
oc describe hpa web-hpa

# View HPA events
oc get events --field-selector involvedObject.name=web-hpa
```

Vertical Pod Autoscaler (VPA)

Vertical Pod Autoscaler automatically adjusts CPU and memory requests and limits for containers based on historical usage data.

How VPA Works

VPA monitors pod resource usage and recommends or automatically updates resource requests and limits:

1. **Metrics Collection**: Collects resource usage metrics from pods
2. **Analysis**: Analyzes historical usage patterns
3. **Recommendations**: Generates resource recommendations
4. **Updates**: Optionally updates pod specifications automatically

VPA Modes

VPA operates in different modes:

- **Off**: VPA only provides recommendations, doesn't make changes
- **Initial**: VPA sets resource requests on pod creation only
- **Auto**: VPA updates resource requests on existing pods (requires pod recreation)

- **Recreate**: VPA recreates pods to apply new resource settings

Creating a VPA

Create a VPA to manage resource requests:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: web-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-deployment
  updatePolicy:
    updateMode: "Auto"
  resourcePolicy:
    containerPolicies:
    - containerName: app
      minAllowed:
        cpu: 100m
        memory: 128Mi
      maxAllowed:
        cpu: 2
        memory: 4Gi
      controlledResources: ["cpu", "memory"]
```

Key parameters:

- `updateMode`: How VPA applies recommendations (Off, Initial, Auto, Recreate)
- `minAllowed`: Minimum resource values to set
- `maxAllowed`: Maximum resource values to set
- `controlledResources`: Which resources VPA should manage

VPA Recommendations

View VPA recommendations:

```
# View VPA status and recommendations
oc get vpa web-vpa -o yaml

# Get recommendation details
oc describe vpa web-vpa
```

VPA recommendations include:

- **Target**: Recommended values for optimal performance
- **Lower Bound**: Minimum values to maintain performance
- **Upper Bound**: Maximum values observed
- **Uncapped Target**: Recommendations without min/max constraints

VPA Resource Policies

Control which containers VPA manages:

```
resourcePolicy:
  containerPolicies:
  - containerName: app
    mode: Auto
    minAllowed:
      cpu: 100m
      memory: 128Mi
    maxAllowed:
      cpu: 2
      memory: 4Gi
  - containerName: sidecar
    mode: Off
```

VPA Limitations

Important considerations when using VPA:

- VPA cannot update resources of running pods; it requires pod recreation
- VPA recommendations are based on historical data and may not reflect sudden changes
- Using VPA with HPA requires careful coordination
- VPA may cause pod churn when updating resources

Interaction Between Components

> Descheduling, Horizontal Pod Autoscaler (HPA), and Vertical Pod Autoscaler (VPA) can interfere with each other if all are deployed simultaneously. Understanding their interactions is crucial for stable cluster operation.

Potential Conflicts

HPA and VPA

HPA and VPA both manage pod resources but in different ways:

- **HPA**: Changes replica count based on resource utilization
- **VPA**: Changes resource requests/limits per pod

If both are active on the same workload:

- VPA may adjust resource requests, affecting utilization percentages that HPA monitors
- HPA may scale replicas up/down, changing the workload that VPA analyzes
- This can create feedback loops and unstable scaling behavior

Best practice: Use either HPA or VPA for a given workload, not both simultaneously.

Descheduler and HPA

Descheduler and HPA can conflict:

- **Descheduler**: Moves pods to optimize cluster balance
- **HPA**: Scales replica count based on metrics

Potential issues:

- Descheduler may move pods that HPA is monitoring, affecting metrics
- HPA scaling decisions may conflict with descheduler's rebalancing efforts
- Rapid scaling combined with descheduling can cause pod churn

Descheduler and VPA

Descheduler and VPA interactions:

- **Descheduler**: Moves pods between nodes
- **VPA**: Updates pod resource specifications

Potential issues:

- VPA may recommend resource changes that trigger descheduling
- Descheduler may move pods that VPA is analyzing
- Pod recreation from VPA updates can interfere with descheduling decisions

Coordination Strategies

If you need to use multiple components:

Use HPA with Descheduler

- Configure HPA with longer stabilization windows to reduce rapid scaling
- Set descheduler to run less frequently
- Use descheduler strategies that don't conflict with HPA metrics

Use VPA in Recommendation Mode

- Run VPA in "Off" mode to get recommendations without automatic updates
- Manually review and apply recommendations during maintenance windows

- Avoid using VPA Auto mode with descheduler

Separate Workloads

- Use HPA for stateless workloads that benefit from horizontal scaling
- Use VPA for stateful workloads or workloads that can't scale horizontally
- Use descheduler for cluster optimization, not workload-specific scaling

Monitoring Interactions

Monitor all components to detect conflicts:

```
# Monitor HPA scaling events
oc get events --field-selector involvedObject.kind=HorizontalPodAutoscaler

# Monitor VPA recommendations
oc get vpa --all-namespaces

# Monitor descheduler activity
oc logs -n kube-system -l app=descheduler

# Watch pod churn
oc get pods -w
```

Best Practices

- Use HPA for workloads that can scale horizontally (stateless applications)
- Use VPA for workloads that need resource optimization but can't scale horizontally
- Avoid using HPA and VPA on the same workload simultaneously
- Configure descheduler with appropriate intervals to avoid excessive pod movement
- Set HPA stabilization windows to prevent rapid scaling oscillations
- Monitor all autoscaling components to detect conflicts early
- Test autoscaling configurations in non-production environments first
- Use Pod Disruption Budgets to protect critical workloads during scaling operations
- Review VPA recommendations before enabling Auto mode
- Coordinate descheduling with application maintenance windows when possible

Exercises

Exercise 1: Horizontal Pod Autoscaler

In this exercise, you will create and configure a Horizontal Pod Autoscaler (HPA) to automatically scale a deployment based on CPU utilization.

Step 1: Create a Deployment

First, create a deployment that will be scaled by the HPA:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: nginx:latest
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 256Mi
```

Apply the deployment:

```
oc apply -f web-deployment.yaml
oc get deployment web-deployment
```

Step 2: Create an HPA

Create a Horizontal Pod Autoscaler that scales the deployment based on CPU utilization:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-deployment
  minReplicas: 2
```

```
  maxReplicas: 4
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Apply the HPA:

```
oc apply -f web-hpa.yaml
oc get hpa web-hpa
```

Step 3: Monitor HPA Status

Monitor the HPA to see its current status and scaling decisions:

```
# View HPA status
oc get hpa web-hpa

# Get detailed HPA information
oc describe hpa web-hpa

# Watch HPA and deployment replicas
oc get hpa web-hpa -w
oc get deployment web-deployment -w
```

Observe the current replica count and target metrics.

Step 4: Generate Load

Generate CPU load on the pods to trigger scaling. In a separate terminal, run:

```
# Get a pod name
POD_NAME=$(oc get pods -l app=web -o jsonpath='{.items[0].metadata.name}')

# Generate CPU load (this will run in the background)
oc exec -it $POD_NAME -- sh -c "while true; do dd if=/dev/zero of=/dev/null; done" &
```

Monitor the HPA and deployment to observe scaling behavior:

```
# Watch HPA scaling
oc get hpa web-hpa -w

# Check deployment replica count
```

```
oc get deployment web-deployment
```

Step 5: Observe Scaling Behavior

After a few minutes, observe how the HPA responds:

```
# View HPA events
oc describe hpa web-hpa

# Check current replica count
oc get pods -l app=web

# View HPA metrics
oc get hpa web-hpa -o yaml
```

Note how the replica count changes based on CPU utilization.

Step 6: Stop Load and Observe Scale-Down

Stop the load generation and observe the scale-down behavior:

```
# Stop the load generation process
oc exec -it $POD_NAME -- pkill dd

# Monitor scale-down
oc get hpa web-hpa -w
oc get deployment web-deployment -w
```

Observe the stabilization window and scale-down policies in action.

Step 7: Clean Up

Remove the test resources:

```
oc delete hpa web-hpa
oc delete deployment web-deployment
```

Exercise 2: Vertical Pod Autoscaler

In this exercise, you will create and configure a Vertical Pod Autoscaler (VPA) to automatically adjust CPU and memory requests and limits for containers based on historical usage.

Step 1: Create a Deployment

Create a deployment that will be managed by the VPA:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpa-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: vpa-app
  template:
    metadata:
      labels:
        app: vpa-app
    spec:
      containers:
      - name: app
        image: nginx:latest
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 256Mi
```

Apply the deployment:

```
oc apply -f vpa-deployment.yaml
oc get deployment vpa-deployment
oc get pods -l app=vpa-app
```

Step 2: Create a VPA in Recommendation Mode

> Creating a VPA requires administrator privileges because VPA is a cluster-scoped resource, not namespace-scoped. Please ask your instructor to create the VPA for you.

Create a VPA in "Off" mode to view recommendations without automatic updates:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: vpa-recommender
  do-not-deploy: "true"
spec:
  targetRef:
    apiVersion: apps/v1
```

```
    kind: Deployment
    name: vpa-deployment
  updatePolicy:
    updateMode: "Off"
  resourcePolicy:
    containerPolicies:
    - containerName: app
      minAllowed:
        cpu: 50m
        memory: 64Mi
      maxAllowed:
        cpu: 2
        memory: 4Gi
      controlledResources: ["cpu", "memory"]
```

The instructor will apply the VPA. After it's created, you can view it:

```
# View the VPA (instructor will create it)
oc get vpa vpa-recommender
```

Step 3: View VPA Recommendations

After the VPA has collected some usage data, view its recommendations:

```
# View VPA status
oc get vpa vpa-recommender -o yaml

# Get detailed VPA information
oc describe vpa vpa-recommender
```

The VPA will show recommendations including:
* **Target**: Recommended values for optimal performance
* **Lower Bound**: Minimum values to maintain performance
* **Upper Bound**: Maximum values observed

Step 4: Generate Load and Monitor

Generate some load on the pods to provide usage data for VPA analysis:

```
# Get a pod name
POD_NAME=$(oc get pods -l app=vpa-app -o jsonpath='{.items[0].metadata.name}')

# Generate some CPU load
oc exec -it $POD_NAME -- sh -c "for i in {1..100}; do dd if=/dev/zero of=/dev/null
bs=1M count=10; done"
```

Wait a few minutes, then check the VPA recommendations again:

```
oc describe vpa vpa-recommender
```

Observe how the recommendations may change based on actual usage patterns.

Step 5: Create VPA in Initial Mode

This step also requires administrator privileges. Please ask your instructor to create this VPA.

Create a new VPA in "Initial" mode that sets resource requests only when pods are first created:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: vpa-initial
  do-not-deploy: "true"
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: vpa-deployment
  updatePolicy:
    updateMode: "Initial"
  resourcePolicy:
    containerPolicies:
    - containerName: app
      minAllowed:
        cpu: 50m
        memory: 64Mi
      maxAllowed:
        cpu: 2
        memory: 4Gi
      controlledResources: ["cpu", "memory"]
```

```
# The instructor will delete the previous VPA and create the new one
# You can delete and recreate the deployment to see Initial mode in action
oc delete deployment vpa-deployment
oc apply -f vpa-deployment.yaml
```

Check the new pods to see if VPA has set resource requests:

```
oc get pods -l app=vpa-app -o yaml | grep -A 10 resources
oc describe pod -l app=vpa-app
```

Step 6: Compare Resource Settings

Compare the original resource settings with VPA recommendations:

```
# View VPA recommendations
oc get vpa vpa-initial -o yaml | grep -A 20 recommendation

# View actual pod resource settings
oc get pod -l app=vpa-app -o jsonpath='{.items[0].spec.containers[0].resources}'
```

Step 7: Clean Up

Remove the test resources. Note that VPA deletion requires administrator privileges:

```
# Ask instructor to delete the VPA
# oc delete vpa vpa-initial

# You can delete the deployment
oc delete deployment vpa-deployment
```

Exercise 3: Descheduler Operator

In this exercise, you will examine the Descheduler Operator configuration and understand how descheduling profiles work. Note that creating or modifying the Descheduler requires administrator privileges.

Step 1: View Descheduler Configuration

Check if the Descheduler Operator is installed and view its configuration:

```
# View Descheduler custom resources (requires admin privileges to create, but you can
view)
oc get kubedescheduler -n openshift-kube-descheduler-operator

# View the cluster Descheduler configuration
oc get kubedescheduler cluster -n openshift-kube-descheduler-operator -o yaml
```

If the Descheduler is configured, you'll see the active profiles and settings.

Step 2: Examine Descheduler Profiles

View the profiles that are enabled in the Descheduler configuration:

```
# View enabled profiles
oc get kubedescheduler cluster -n openshift-kube-descheduler-operator -o
jsonpath='{.spec.profiles}'
```

```
# View full Descheduler spec
oc get kubedescheduler cluster -n openshift-kube-descheduler-operator -o
jsonpath='{.spec}' | jq .
```

Note which profiles are active. The available profiles include:
* AffinityAndTaints
* TopologyAndDuplicates
* SoftTopologyAndDuplicates
* LifecycleAndUtilization
* LongLifecycle
* CompactAndScale
* KubeVirtRelieveAndMigrate
* EvictPodsWithPVC
* EvictPodsWithLocalStorage

Step 3: View Descheduler Settings

Examine the Descheduler configuration settings:

```
# View descheduling interval
oc get kubedescheduler cluster -n openshift-kube-descheduler-operator -o
jsonpath='{.spec.deschedulingIntervalSeconds}'

# View descheduler mode (Predictive or Automatic)
oc get kubedescheduler cluster -n openshift-kube-descheduler-operator -o
jsonpath='{.spec.mode}'

# View eviction limits
oc get kubedescheduler cluster -n openshift-kube-descheduler-operator -o
jsonpath='{.spec.evictionLimits}'
```

Step 4: View Descheduler Operator Status

Check the status of the Descheduler Operator:

```
# View Descheduler Operator deployment
oc get deployment -n openshift-kube-descheduler-operator

# View Descheduler pods
oc get pods -n openshift-kube-descheduler-operator

# View Descheduler Operator logs (if accessible)
oc logs -n openshift-kube-descheduler-operator -l app=descheduler-operator --tail=50
```

Step 5: Understand Descheduler Behavior

Create a deployment to observe how descheduling might affect it:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: descheduler-test
spec:
  replicas: 3
  selector:
    matchLabels:
      app: descheduler-test
  template:
    metadata:
      labels:
        app: descheduler-test
    spec:
      containers:
      - name: app
        image: nginx:latest
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
```

```
oc apply -f descheduler-test.yaml
oc get pods -l app=descheduler-test -o wide
```

Monitor the pods to see their distribution across nodes. If the Descheduler is active with certain profiles, it may move pods to optimize cluster balance.

Step 6: Check for Descheduling Events

Look for descheduling-related events:

```
# View events related to descheduling
oc get events --all-namespaces --field-selector reason=Descheduled

# View recent events for your test deployment
oc get events --field-selector involvedObject.name=descheduler-test

# Watch pod movements
oc get pods -l app=descheduler-test -o wide -w
```

Note: If the Descheduler is in Predictive mode, it will simulate evictions but not actually move pods. Only Automatic mode performs actual evictions.

Step 7: Example Descheduler Configuration

Creating or modifying the Descheduler requires administrator privileges. This

> example shows what a Descheduler configuration might look like:

```
apiVersion: operator.openshift.io/v1
kind: KubeDescheduler
metadata:
  name: cluster
  namespace: openshift-kube-descheduler-operator
  do-not-deploy: "true"
spec:
  managementState: Managed
  deschedulingIntervalSeconds: 3600
  profiles:
  - AffinityAndTaints
  - TopologyAndDuplicates
  mode: Automatic
  evictionLimits:
    total: 10
    node: 5
```

This configuration would:
* Run descheduling every hour (3600 seconds)
* Use the AffinityAndTaints and TopologyAndDuplicates profiles
* Actually evict pods (Automatic mode)
* Limit evictions to 10 total and 5 per node per run

Step 8: Clean Up

Remove the test deployment:

```
oc delete deployment descheduler-test
```

Module 4: Image Distribution and Optimization

Content

> This module covers mostly administrator operations and is provided for informational purposes. Many of the configurations and operations described require cluster administrator privileges and should not be executed by regular users. The content is intended to help you understand how image management works in OpenShift, but you should consult with your cluster administrator for actual implementation.

Efficient image distribution and management are critical for optimized deployments in OpenShift. This module explores how image placement and image distribution mechanisms impact deployment performance, resource utilization, and application availability. Understanding these concepts helps ensure fast pod startup times and efficient cluster resource usage.

Why Image Management Matters

Image distribution and placement directly impact several critical aspects of cluster operations:

Deployment Speed

Fast image pulls are essential for rapid application deployment and scaling. When images are already cached on nodes, pods can start immediately without waiting for network transfers. This is especially important for:

- Horizontal scaling operations where multiple pods need to start simultaneously
- Rolling updates that require new pod instances
- Disaster recovery scenarios requiring rapid pod recreation
- Auto-scaling events that need immediate capacity

Network Bandwidth

Efficient image distribution reduces network congestion and bandwidth costs:

- Pre-pulled images eliminate repeated downloads of the same image
- Image caching reduces registry load and network traffic
- Strategic image placement minimizes cross-zone or cross-region transfers
- Reduced bandwidth usage lowers infrastructure costs

Resource Utilization

Proper image management optimizes cluster resources:

- Node disk space is used efficiently through image garbage collection
- CPU and memory are preserved by avoiding unnecessary image pulls
- Network resources are conserved for application traffic
- Storage costs are minimized through effective image lifecycle management

Application Availability

Image availability directly affects application reliability:

- Image pull failures can prevent pods from starting
- Network issues can delay or block deployments
- Registry outages can impact application updates
- Image placement strategies ensure images are available when needed

Image Pull Configuration

Control how images are pulled and authenticated in your cluster.

Image Pull Policies

Control how images are pulled at the pod level:

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: app
    image: myapp:latest
    imagePullPolicy: Always
```

Available policies:
* **Always**: Always pull the image, even if it exists locally
* **IfNotPresent**: Only pull if the image doesn't exist locally (default for tags other than `latest`)
* **Never**: Never pull the image; it must exist locally

Image Pull Secrets

Configure authentication for private registries:

```
# Create an image pull secret
oc create secret docker-registry myregistrykey \
  --docker-server=registry.example.com \
  --docker-username=myuser \
  --docker-password=mypassword \
```

```
  --docker-email=myuser@example.com

# Link secret to service account
oc secrets link default myregistrykey --for=pull
```

```
apiVersion: v1
kind: Pod
metadata:
  name: private-image-pod
spec:
  imagePullSecrets:
  - name: myregistrykey
  containers:
  - name: app
    image: registry.example.com/private/app:latest
```

Image Placement and Distribution

Strategic image placement ensures images are available where and when they're needed.

Node Image Caching

OpenShift caches images on nodes to speed up subsequent deployments:

```
# View images on a node (requires node access)
oc debug node/<node-name> -- chroot /host crictl images

# Check image cache usage
oc debug node/<node-name> -- chroot /host df -h /var/lib/containers
```

Image caching provides:
* Faster pod startup times
* Reduced network bandwidth usage
* Lower registry load
* Improved deployment reliability

Image Garbage Collection

OpenShift automatically manages image storage through garbage collection:

```
# View kubelet configuration for image GC
oc get kubeletconfig -o yaml

# Check node disk usage
oc adm top nodes
```

Garbage collection settings control:
* When images are removed from nodes
* Disk usage thresholds that trigger cleanup
* Minimum age before images can be garbage collected

Image Distribution Strategies

Different strategies optimize image distribution:

Pre-pulling Images

Pre-pull critical images before they're needed:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: image-prepull
spec:
  selector:
    matchLabels:
      name: image-prepull
  template:
    metadata:
      labels:
        name: image-prepull
    spec:
      initContainers:
      - name: prepull
        image: registry.example.com/myapp:v1.2.3
        command: ["echo", "Image pre-pulled"]
      containers:
      - name: pause
        image: k8s.gcr.io/pause:3.1
```

Image Puller

The Image Puller operator provides a simpler way to manage pre-pulling images across cluster nodes. While you can manually create DaemonSets to pre-pull images, the Image Puller operator makes it simpler to maintain multiple images and ensures images are available before workloads need them.

How Image Puller Works

The Image Puller runs as a DaemonSet with one pod per node. Each pod:

1. **Pulls Specified Images**: Downloads images listed in the configuration
2. **Caches on Node**: Stores images in the node's image cache
3. **Maintains Availability**: Keeps images available for pod scheduling

4. **Updates Automatically**: Can be configured to pull updated images

Image Puller Configuration

Configure the Image Puller through a custom resource:

```
apiVersion: apps.openshift.io/v1
kind: KubernetesImagePuller
metadata:
  name: cluster-image-puller
  namespace: openshift-image-puller
spec:
  images: "registry.example.com/myapp:v1.2.3,registry.example.com/nginx:latest"
  nodeSelector:
    node-role.kubernetes.io/worker: ""
```

Configuration options:

- **images**: Comma-separated string of images to pre-pull
- **nodeSelector**: Which nodes should pull images

Viewing Image Puller Status

Monitor the Image Puller to ensure images are being pulled:

```
# View Image Puller custom resource
oc get kubernetesimagepuller -n openshift-image-puller

# View Image Puller DaemonSet
oc get daemonset -n openshift-image-puller

# View Image Puller pods
oc get pods -n openshift-image-puller

# View Image Puller logs
oc logs -n openshift-image-puller -l app=image-puller --tail=50
```

Image Puller Benefits

Using the Image Puller provides several advantages:

- **Faster Deployments**: Images are already cached when pods need them
- **Reduced Network Congestion**: Images are pulled during off-peak times
- **Improved Reliability**: Images are verified before workloads need them
- **Predictable Performance**: Eliminates variable image pull times
- **Cost Optimization**: Reduces bandwidth costs by pulling during low-traffic periods

Image Puller Best Practices

- Pre-pull only critical images that are frequently used
- Use specific image tags rather than `latest` for predictability
- Configure appropriate pull policies based on update frequency
- Monitor Image Puller pods to ensure successful pulls
- Update image lists when new versions are released
- Use node selectors to target specific node types
- Consider image sizes and node storage capacity

Image Puller and Deployment Optimization

The Image Puller directly supports optimized deployments by:

- **Eliminating Pull Delays**: Pods start immediately without waiting for image downloads
- **Supporting Rapid Scaling**: Multiple pods can start simultaneously with cached images
- **Enabling Fast Rollbacks**: Previous image versions remain cached
- **Reducing Deployment Failures**: Images are verified before deployment
- **Improving User Experience**: Applications become available faster

Registry Mirrors

Use registry mirrors to reduce latency:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: registry-mirror
spec:
  config:
    ignition:
      version: 3.2.0
    storage:
      files:
      - path: /etc/containers/registries.conf.d/mirror.conf
        mode: 0644
        contents:
          source: data:text/plain;base64,...
```

Lean Base Images

Using lean, optimized base images significantly improves deployment performance and resource utilization. Lean base images are minimal container images that contain only the essential components needed to run applications.

Benefits of Lean Base Images

Lean base images provide several advantages:

- **Faster Image Pulls**: Smaller images download faster, reducing deployment time
- **Reduced Storage Usage**: Less disk space required on nodes
- **Lower Security Surface**: Fewer components mean fewer potential vulnerabilities
- **Faster Container Startup**: Minimal images start containers more quickly
- **Reduced Network Bandwidth**: Smaller images consume less bandwidth during pulls
- **Better Cache Efficiency**: Smaller images improve layer caching effectiveness

Internal Team Base Images

Many organizations provide standardized base images through internal teams. These images offer:

- **Security Hardening**: Pre-configured with security best practices
- **Compliance**: Built to meet organizational compliance requirements
- **Standardization**: Consistent base images across all applications
- **Maintenance**: Regularly updated and patched by dedicated teams
- **Optimization**: Optimized for your specific infrastructure and use cases
- **Support**: Internal support and documentation

Choosing Base Images

When selecting base images, consider:

- **Size**: Prefer minimal images (Alpine, distroless, or internal lean images)
- **Security**: Use images from trusted sources with regular security updates
- **Compatibility**: Ensure base images are compatible with your application requirements
- **Maintenance**: Choose images with active maintenance and support
- **Internal Options**: Prefer internal team-provided images when available

Image Size Impact

Image size directly affects deployment performance:

```
# Compare image sizes
oc image info registry.example.com/myapp:latest | grep -i size
oc image info registry.example.com/ubi9/ubi:latest | grep -i size
```

A lean base image (50-100MB) compared to a full OS image (500MB-1GB) can:
* Reduce pull time by 80-90%
* Decrease storage requirements by 80-90%
* Improve cache hit rates

* Enable faster scaling operations

Multi-stage Builds for Lean Images

Use multi-stage builds to create lean production images:

```
# Build stage with full toolchain
FROM registry.example.com/build-tools:latest AS builder
WORKDIR /build
COPY . .
RUN make build

# Production stage with lean base image
FROM alpine:latest
WORKDIR /app
COPY --from=builder /build/bin/app /app/app
USER 1001
CMD ["/app/app"]
```

This approach:
* Keeps build tools out of production images
* Uses lean base images for runtime
* Minimizes final image size
* Reduces security exposure

Best Practices for Lean Images

- Use internal team-provided base images when available
- Prefer minimal base images (Alpine, distroless, or internal equivalents)
- Remove unnecessary packages and files
- Use multi-stage builds to separate build and runtime environments
- Regularly update base images to include security patches
- Document which base images are approved for use
- Monitor image sizes and optimize continuously

Monitoring Image Distribution

Monitor image-related metrics to optimize distribution:

```
# View image pull events
oc get events --field-selector reason=Failed

# Check pod image pull status
oc describe pod <pod-name> | grep -i image

# Monitor node image cache usage
```

```
oc adm top nodes
```

Key metrics to monitor:

- Image pull success/failure rates
- Image pull duration
- Node image cache utilization
- Registry request rates
- Image garbage collection frequency

Best Practices

- Use lean base images, especially those provided by internal teams
- Prefer minimal base images (Alpine, distroless, or internal equivalents) over full OS images
- Pre-pull critical images using Image Puller
- Use specific image tags instead of `latest` for production
- Monitor image pull performance and failures
- Configure appropriate image pull policies
- Implement image pull secrets for private registries
- Use multi-stage builds to create lean production images
- Regularly review and update image lists
- Monitor node disk usage and image cache
- Test image distribution strategies in non-production first
- Use registry mirrors for reduced latency
- Configure image garbage collection thresholds appropriately
- Document approved base images and image sources